

Document Summary TASK-7F12F441

Analysis & Generated Response

Certainly! Below is a simple example of an encryption algorithm in Python using the Caesar cipher for demonstration purposes. This code includes basic error handling and ensures that the input message is within a reasonable length to prevent errors.

```
```python
def caesar_encrypt(message, shift):
 if not isinstance(message, str) or len(message) == 0:
 raise ValueError("Message must be a non-empty string.")

 encrypted_message = ""
 for char in message:
 if char.isalpha():
 is_uppercase = char.isupper()
 base = ord('A') if is_uppercase else ord('a')
 shifted_char = chr((ord(char) - base + shift) % 26 + base)
 encrypted_message += shifted_char
 else:
 encrypted_message += char

 return encrypted_message

Example usage
message = "Hello, World!"
shift = 3

encrypted = caesar_encrypt(message, shift)
print(f"Original: {message}")
print(f"Encrypted: {encrypted}")
```
```

Explanation:

1. **Function Definition:** The `caesar\_encrypt` function takes two parameters:

- `message`: A string to be encrypted.
- `shift`: An integer representing the number of positions each letter in the message should be shifted.

2. Input Validation:

- Checks if the input is a non-empty string. If not, raises a `ValueError`.

3. Encryption Loop:

- Iterates through each character in the input message.
- For alphabetic characters (both uppercase and lowercase), it shifts them by the specified number of positions.
- Uses ASCII values to determine if the character is uppercase or lowercase.
- Applies modulo operation to wrap around the alphabet if necessary.
- Converts the shifted ASCII value back to a character.

4. Non-Alphabetic Characters:

- Non-alphabetic characters are added directly to the encrypted message without any modification.

Usage:

- The example usage demonstrates encrypting the string "Hello, World!" with a shift of 3 positions.
- The original and encrypted messages are printed to show the result.

This code is simple but can be expanded with additional features such as key management, padding for longer messages, or support for different encryption algorithms.